

TECHNICAL DESIGN MEMO

轮腿底盘倒地检测与 自起模块重构设计方案

以四元数推导机体系世界竖直方向为主判据

项目 embedded-2026-h7 / wheel_legged

适用基线 当前默认参数分支：Infantry3（同时给出多机型参数迁移方式）

文档状态 设计评审稿

版本 V1.0

日期 2026-07-23

目标读者 底盘控制、状态机、嵌入式与联调测试人员

核心结论

建议用 IMU 四元数计算“世界竖直方向在机体系下的单位向量”作为机身姿态主观测量；加速度计只在低动态时做一致性校验，腿部构型、角速度、数据新鲜度和运行模式分别作为安全条件。检测、决策、自起动作和起立动作必须拆成独立模块，禁止再以单个 posture_valid 同时表达多种含义。

文档导航

章节	内容	评审重点
1-3	背景、结论、当前实现审计	为什么重构；现有行为是否被准确复现
4-7	坐标系、四元数、姿态观测与倒地检测	四元数方向与安装变换；阈值、滞回和时间确认
8-10	状态机、自起序列与起立序列	职责边界、故障锁定、动作安全性
11-13	接口、参数、数据流	能否独立单测；是否消除上一周期反馈
14-17	迁移、测试、验收与风险	影子模式、回滚条件、上线门槛
附录	公式、测试姿态、代码定位	现场验证可直接照表执行

1. 背景与目标

当前倒地检测、底盘控制、自起动作和三段式起立逻辑分散在 control.cc、input.cc、chassis_fsm.cc 与 chassis.cc 中。它们通过 posture_valid、上一周期 Chassis::Update 输出、局部计时变量和若干隐式 latch 互相耦合。该结构能够运行，但难以准确回答“现在为什么进入恢复”、“动作是否已经超时”“是哪一种倒地”和“传感器失效时应该怎样处理”。

本方案的目标不是单纯把欧拉角替换成四元数，而是建立一条可验证的数据链：传感器原始量 → 坐标统一 → 姿态观测 → 倒地事件 → 主状态机 → 自起序列 → 起立序列 → 力矩合成。每一层有清晰输入、输出、超时、故障语义与测试方法。

1.1 设计目标

- 避免欧拉角奇异性、角度绕回和 pitch/roll 耦合对倒地判断的影响。
- 用一个与 yaw 无关、几何意义明确的量描述机身倾斜，并支持前、后、左、右、倒扣和斜向分类。
- 把“机身是否直立”“腿是否处于安全构型”“IMU 是否可信”“是否允许自动恢复”拆开表达。
- 自起失败后进入显式锁定或受控重试，杜绝超时停机只维持一个控制周期。

- 支持影子模式、日志回放、单元测试与逐级实机验证，降低一次性切换风险。
- 保留当前可用动作参数和不同机器人 variant 的差异，先做行为等价拆分，再做算法升级。

1.2 非目标

- 本阶段不重新设计腿部动力学、VMC/Jacobian 或电机底层闭环。
- 本阶段不承诺仅靠一组初始阈值覆盖所有场地；参数需要日志与实机标定。
- 本阶段不把加速度计单独作为动态场景的最终姿态解算器。

2. 推荐方案摘要

推荐判据

以归一化四元数 q 计算机体系下的世界上方向 $u_b = [u_x, u_y, u_z]^T$ 。 u_z 是机体上轴与世界上轴夹角的余弦： $u_z \approx 1$ 为直立， $u_z \approx 0$ 为侧躺/前后躺， $u_z \approx -1$ 为倒扣。倒地进入和直立退出分别使用不同阈值与持续时间； u_x 、 u_y 只在倒地确认后锁定用于方向分类。

信息源	角色	是否主判据	主要原因
四元数 / 旋转矩阵	机身倾斜与倒地方向	是	不受 yaw 影响；无欧拉角奇异和绕回
陀螺仪	静稳确认、方向锁定时机、动作超速保护	辅助	区分“已直立”与“正在高速穿过直立”
加速度计	低动态一致性校验、冲击/自由落体提示	辅助	动态时含线加速度，不能始终等同重力
腿角/腿长/关节状态	构型安全与起立完成条件	独立安全条件	机身直立不代表腿已可承载
模式/遥控/故障状态	自动恢复许可	门控条件	跳跃、台阶、维护或失联场景需要不同策略

2.1 初始检测策略（待数据标定）

事件	建议初值	含义
普通倒地进入	倾角 45°–50°，持续 180–250 ms	抑制急加减速、过坎和瞬时冲击误触发
严重倒地快速进入	倾角 75°–80°，持续 50–100 ms	明显侧躺/前后躺时缩短响应
恢复直立退出	倾角 25°–30° 内，持续 300–500 ms	形成滞回，防止边界抖动
直立角速度	$ \omega < 0.5\text{--}1.0\text{ rad/s}$	避免高速摆动穿过直立时提前结束
加速度低动态窗口	$ a -g < 0.15g\text{--}0.25g$	仅在此窗口检查加速度方向一致性
IMU 新鲜度	按采样周期设 2–5 帧上限	超时或非有限值进入传感器故障，不执行方向性动作

这些数值是工程起点，不是最终标定值。推荐先记录当前方案与新方案的并行结果，使用正常行驶、急加减速、旋转、跳跃、台阶和人工翻倒日志生成混淆矩阵后再冻结参数。

3. 当前代码实现审计

3.1 现有数据与状态流

图 1 当前循环中的关键依赖（概念化）



现状	代码位置	问题与影响
posture_valid 同时检查机身欧拉角和腿摆角	app/targets/wheel_legged/chassis.cc:307–318	一个布尔量混合“机身倒地”和“腿构型越界”，导致恢复原因、动作选择和测试无法分离。
FSM 输入使用上一周期 chassis_control_output	control.cc:369, 420–446; input.cc:739–776	形成隐式一拍延迟和循环依赖；事件发生在哪一周期不直观，重构容易产生时序差异。
FallCheck 与 SelfRight 输出相同 recovery_enable	chassis_fsm.cc:181–186, 439–456	物理恢复分支由 chassis.cc 的 posture_valid 立即触发，220 ms 确认阶段并未真正延迟动作。
自起超时转 Disabled	chassis_fsm.cc:447–451	若上层 domain 仍启用，Disabled 可能下一周期又离开；失败状态没有持久锁定和明确复位条件。
起立 phase 与恢复动作共存于大函数	chassis.cc:367–453, 678–871	phase、恢复方向、gimbal 让位、腿控制和力矩输出交织，难以独立回放和单测。
IMU 轴存在显式符号/交换	include/actuators.hpp (roll/pitch/gyro 映射)	说明传感器系与机体系并非天然一致；四元数接入前必须确定固定安装旋转，而不能只套公式。
库已提供 w,x,y,z 四元数	libs/librm/.../hipnuc_imu.hpp:95–98	具备直接计算竖直向量的输入条件，但当前底盘反馈接口尚未把它作为一等数据传递。

3.2 当前起立逻辑概括

阶段	当前意图	当前主要完成条件	重构建议
Phase 0	腿摆到初始目标；恢复来源可跳过部分准备	腿摆角接近目标	显式定义 Full / RetractAtZero 启动模式
Phase 1	收腿到低位	腿长达到目标附近	增加连续稳定周期、阶段超时和传感器有效条件
Phase 2	腿长保持低位，腿摆角斜坡回零	摆角误差满足容差	把 ramp 改为按 dt 的速率限制，而非每周期常数
Phase 3	完成并交回正常控制	phase 置完成	由 StandupSequence 输出 done，主 FSM 决定状态转移

3.3 当前实现需要优先修正的语义问题

- 1. RecoveryFallCheck 应是纯观察/等待确认状态，不得提前输出会导致机器人动作的恢复指令。
- 2. upright_stable 必须是持续满足后的状态，而不是姿态从 invalid 到 valid 的单次边沿。
- 3. 超时失败必须进入持久 RecoveryFailed/Lockout，直到明确复位、重新使能或人工确认。
- 4. posture_valid 应拆为 body_upright、leg_configuration_safe、sensor_valid 等独立字段。
- 5. 未使用或语义漂移的参数/字段应在行为等价阶段建立清单，再逐个删除，避免把“看似无用但被调参依赖”的量一次性清掉。

4. 坐标系与四元数约定

上线前置条件：先确定约定

必须确认 IMU 输出四元数表示 R_{wb} （机体系向量旋到世界系）还是 R_{bw} （世界系向量旋到机体系），并确认四元数顺序是 $[w,x,y,z]$ 、右手系方向以及 IMU 安装坐标到车体坐标的固定旋转。仅凭变量名或现有 Euler 符号映射不能替代六面静置试验。

4.1 定义

- 世界系 W ：定义 z_W 向上；世界上方向 $e_z = [0,0,1]^T$ 。若系统使用重力向下，则 $\hat{g}_W = [0,0,-1]^T$ ，二者只差负号。
- 机体系 B ：必须在项目中固定 x_B 、 y_B 、 z_B 的物理方向，建议 z_B 为车体上方。
- IMU 传感器系 S ：由器件丝印/安装决定；与机体系之间存在常量旋转 R_{BS} 。
- 最终需要的是 u_B ：世界上方向在机体系 B 中的表示，而不是传感器系中的表示。

4.2 从旋转矩阵得到竖直向量

若四元数对应 R_{WB} ，即 $v_W = R_{WB} v_B$ ，则世界上方向在机体系中为：

$$u_B = R_{WB}^T \cdot e_z$$

因此 u_B 等于 R_{WB} 的第三行。

若四元数对应 R_{BW} ，即 $v_B = R_{BW} v_W$ ，则：

$$u_B = R_{BW} \cdot e_z$$

因此 u_B 等于 R_{BW} 的第三列。

若先在 IMU 系得到 u_S ，则安装变换为 $u_B = R_{BS} \cdot u_S$ 。推荐把 R_{BS} 作为唯一的常量配置，避免在不同文件中继续散落 x/y 交换与负号。

4.3 从 $[w,x,y,z]$ 四元数直接计算

对归一化四元数 $q = [w,x,y,z]$ ，若 q 表示 R_{WB} ，按常见右手 Hamilton 约定，可直接计算：

推荐实现：避免构造完整旋转矩阵

$$\begin{aligned} u_{B.x} &= 2 * (x*z - w*y) \\ u_{B.y} &= 2 * (y*z + w*x) \\ u_{B.z} &= 1 - 2 * (x*x + y*y) \end{aligned}$$

若 q 表示相反方向，使用共轭 q^* 或改用对应矩阵公式。注意 q 与 $-q$ 表示同一旋转，上述二次项结果完全相同；但把 R_{WB} 误当 R_{BW} 时， u_x 、 u_y 的符号/组合可能改变。 u_z 对转置保持一致，因此仅判断“倾斜程度”可能看不出约定错误，但前后左右分类会错。

4.4 数值处理

```
norm2 = w*w + x*x + y*y + z*z
valid = all_finite(q) && norm2 > min_norm2 && imu_age_ms <= max_age_ms
if valid:
    inv_norm = 1 / sqrt(norm2)
    q = q * inv_norm
    u_B = rotate_inverse(q, [0,0,1])
    u_B = R_BS * u_S           // 若 IMU 与车体不共轴
    tilt_cos = clamp(u_B.z, -1, 1)
    tilt_rad = acos(tilt_cos)   // 仅调试/UI 需要角度时计算
```

实时判定可直接比较余弦，避免 acos ： $\text{tilt} > 50^\circ$ 等价于 $u_z < \cos(50^\circ) \approx 0.643$ 。这既减少计算，也让阈值与单位向量的几何意义保持一致。

5. 六面静置与安装标定

在接入状态机前，先编写仅输出 q 、 u_S 、 u_B 、 $|q|$ 、Euler（仅对照）的影子观测代码。将机器人或 IMU 固定在六个已知姿态，记录稳定均值。期望值取决于项目机体系轴定义，下表以 x_B 向前、 y_B 向左、 z_B 向上为例。

静置姿态	期望 u_B	关键检查
正常直立	$[0, 0, +1]$	u_z 接近 $+1$ ， x/y 接近 0
车头朝上（后躺）	$[+1, 0, 0]$	确认 u_x 符号与“向前”定义
车头朝下（前趴）	$[-1, 0, 0]$	应与上一姿态相反
左侧朝上（右侧躺）	$[0, +1, 0]$	确认 y_B 的正方向
右侧朝上（左侧躺）	$[0, -1, 0]$	应与上一姿态相反
倒扣	$[0, 0, -1]$	u_z 接近 -1

5.1 验证通过标准

- 每个姿态下 $|q|$ 接近 1 ， u_B 的模长接近 1 ；非单位四元数归一化后结果稳定。
- 六面方向均符合预期，不允许只验证直立与倒扣，因为这两种姿态无法暴露全部符号错误。
- 绕机体 z 轴改变 yaw 时， u_B 与 tilt_cos 基本不变。
- q 与 $-q$ 输入产生相同 u_B ；记录中出现四元数符号翻转时检测器不应抖动。
- Euler 仅作为对照，不参与最终通过条件；若二者冲突，以已知物理姿态和坐标定义定位问题。

标定产物

评审后应冻结一份 FrameConvention 文档/头文件：轴方向、四元数顺序、旋转方向、固定安装四元数 q_{BS} 或矩阵 R_{BS} ，以及六面测试记录。之后所有模块只消费统一后的 BodyFrameImu。

6. PostureObserver 设计

PostureObserver 只负责把原始传感器量转换成带质量标志的机体姿态观测，不做状态机决策，也不输出电机命令。它应在每个控制周期最先运行，并由同一周期的 FallDetector 和控制器消费。

6.1 输入与输出

```
struct Quaternionf { float w, x, y, z; };
struct PostureObserverInput {
    Quaternionf quat_sensor;
    Vec3 gyro_sensor_rad_s;
    Vec3 accel_sensor_m_s2;
    uint32_t imu_sample_tick_ms;
};

struct PostureObservation {
    Vec3 up_body;           // 世界上方向在机体系中的表示
    float tilt_cos;         // == up_body.z
    Vec3 gyro_body_rad_s;
    Vec3 accel_body_m_s2;
    float accel_norm_m_s2;
    bool quaternion_valid;
    bool imu_fresh;
    bool accel_low_dynamic;
    uint32_t sample_age_ms;
    PostureFault fault;
};
```

6.2 处理顺序

1. 检查四元数、陀螺仪和加速度计是否为有限值，并检查采样时间戳新鲜度。
2. 检查四元数范数；在允许范围内归一化，过小或异常跳变则标记无效。
3. 按已确认的旋转方向计算 up_sensor，再用固定 R_BS 变换到机体系。
4. 把 gyro 与 accel 使用同一 R_BS 变换，禁止不同传感器量采用不同散落符号规则。
5. 输出 tilt_cos、模长、低动态标志和故障码；不在此层加入倒地阈值或计时。

6.3 质量与故障策略

异常	Observer 输出	下游策略
四元数 NaN/Inf/范数过小	quaternion_valid=false	停止方向性自起；进入传感器故障/安全停机
IMU 数据超时	imu_fresh=false + age	不得沿用最后一次方向无限动作
加速度模长异常	accel_low_dynamic=false	忽略重力方向校验，但不直接否定四元数姿态
瞬时四元数跳变	fault=discontinuity	短时保持/拒绝并计数；达到阈值后故障
安装配置缺失	fault=frame_config	编译或启动失败，禁止默认假设共轴

7. FallDetector 设计

FallDetector 是纯事件检测器：输入同周期 PostureObservation、腿构型与模式上下文，输出稳定的倒地/直立状态、方向和原因。它不决定执行哪套力矩，也不直接切换主 FSM。

7.1 输出语义

```
struct FallDetection {
    bool body_raw_upright;           // 瞬时几何判断，仅供诊断
    bool body_upright_confirmed;     // 通过退出阈值+持续时间+角速度
    bool fall_confirmed;             // 通过进入阈值+持续时间
    bool severe_fall;                // 快速路径
    bool leg_configuration_safe;     // 独立于机身姿态
    bool sensor_valid;
    FallDirection direction;         // Unknown/Front/Back/Left/Right/Inverted/Diagonal
    FallCause cause;                 // BodyTilt/LegGeometry/SensorFault/ExternalMode...
    uint32_t condition_hold_ms;
};
```

7.2 滞回与时间确认

检测器伪代码

```
enter_cos  = cos(fall_enter_angle)      // 例如 cos(50°)=0.643
severe_cos = cos(severe_angle)          // 例如 cos(78°)=0.208
exit_cos   = cos(upright_exit_angle)    // 例如 cos(28°)=0.883

fall_candidate  = sensor_valid && (up_body.z < enter_cos)
severe_candidate = sensor_valid && (up_body.z < severe_cos)
upright_candidate = sensor_valid
                  && (up_body.z > exit_cos)
                  && (norm(gyro_body) < upright_gyro_max)
                  && leg_configuration_safe

fall_confirmed = held(fall_candidate, fall_confirm_ms)
                || held(severe_candidate, severe_confirm_ms)
upright_confirmed = held(upright_candidate, upright_confirm_ms)
```

7.3 倒地方向分类

方向只在 `fall_confirmed` 的跃迁时锁定，避免自启动动作导致 `u_x/u_y` 变化后方向来回翻转。以 `x_B` 向前、`y_B` 向左为例：比较 `|u_x|` 与 `|u_y|`，并设置方向死区/优势比。

条件（示例）	分类	动作含义需由实机确认
<code>u_z < inverted_cos</code>	Inverted	倒扣专用策略或禁止自动动作
<code> u_x > k· u_y 且 <code>u_x < 0</code></code>	Front	前趴
<code> u_x > k· u_y 且 <code>u_x > 0</code></code>	Back	后躺
<code> u_y > k· u_x 且 <code>u_y < 0</code></code>	Left	左侧躺
<code> u_y > k· u_x 且 <code>u_y > 0</code></code>	Right	右侧躺
其余	Diagonal	选择保守动作或先转入最近主方向

上表符号必须用第 5 章六面试验确认。变量名 `Front/Back` 不能直接从公式推断，因为最终还受机体系定义、IMU 安装和机械动作正方向影响。

7.4 加速度计的正确用法

仅使用加速度计判断倒地在静止或低动态时可行，但机器人起步、刹车、跳跃、碰撞和自起动作都会引入线加速度。加速度计测到的是比力，不是任何时刻都等于重力。若直接用 $a/|a|$ 作为竖直方向，可能把急加速误判为倾斜，或在自由落体时得到随机方向。

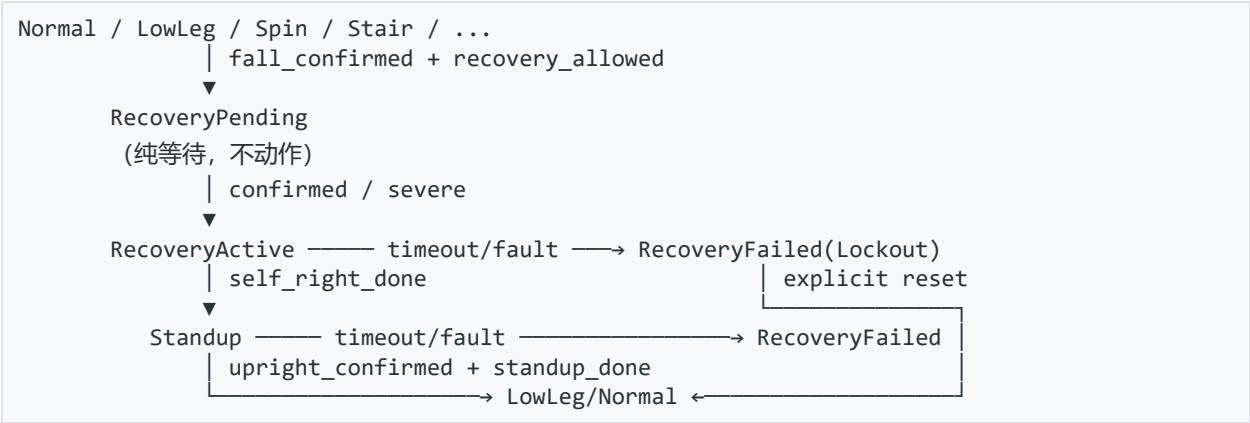
```
low_dynamic = abs(norm(accel) - g) < accel_norm_tolerance
if low_dynamic:
    accel_dir = normalize(accel_body)          // 正负号由器件输出定义确认
    consistency = dot(accel_dir, up_body_or_down_body)
    if consistency < threshold for T ms:
        flag attitude_inconsistency
else:
    do not use accel direction to veto quaternion fall detection
```

结论：不要只用加速度计

加速度计适合低动态校验和冲击/自由落体识别；主姿态仍应使用 IMU 融合后的四元数。这样既避免欧拉角问题，又保留陀螺仪在动态过程中的短期姿态信息。

8. 主状态机重构

图 2 推荐主状态机



状态	允许动作	退出条件	失败策略
RecoveryPending	轮/腿保持安全零输出或既定制动；不得自起	倒地确认、候选消失、禁用/故障	候选消失回原状态
RecoveryActive	运行 SelfRightSequence	序列完成且进入可起立姿态	超时/传感器故障 → Lockout
Standup	运行 StandupSequence	序列完成且 upright_confirmed	阶段超时/关节故障 → Lockout
RecoveryFailed	安全零输出/制动；保持故障码	明确复位或受控重试许可	不得因 domain 仍 enable 自动离开

8.1 恢复许可与模式感知

FallDetector 应始终运行并记录，但 recovery_allowed 由策略层给出。跳跃、台阶攀爬、调试悬挂、遥控失联、急停、IMU 故障和低电压等场景应有明确门控。门控禁止的是自动动作，不应掩盖 fall_confirmed 事实。

8.2 重试策略

- 默认建议：首次失败进入锁定，等待人工复位；调试阶段最安全。

- 若比赛策略需要自动重试，最多 1 次，且必须经过冷却、姿态重新分类和能量/温度检查。
- 重试次数、失败原因、最后方向和各阶段耗时必须遥测可见。

9. SelfRightSequence 设计

SelfRightSequence 只描述动作意图，输出虚拟腿力、虚拟腿摆力矩、轮速/轮矩许可和 gimbal 让位请求；具体电机符号与 Jacobian 映射仍由底层统一完成。这样可以离线回放阶段逻辑，也避免序列代码直接依赖四个关节电机的正负号。

阶段	目标	主要输入	完成条件	超时/保护
Classify	锁定方向与策略	up_body、gyro、腿状态	方向置信度足够	Unknown → 保守停机
GimbalClearance	避免自起与云台干涉	gimbal yaw/ack	进入安全角或收到确认	超时走降级策略或锁定
LegReposition	把腿移到可发力构型	腿角/腿长/关节故障	连续稳定 N ms	关节限位/堵转保护
BodyPush	按方向产生翻身力矩	方向、角速度、u_body	进入接近直立/可起立区	力矩、速度、时间、温度限制
Settle	卸力并等待姿态稳定	u_z、gyro、腿状态	满足交接窗口	未稳定则有限重试或失败

9.1 动作安全原则

- 方向在序列开始时锁定；仅在动作尚未发力且置信度不足时允许重新分类。
- 所有 ramp 使用物理单位/秒并乘 dt，避免控制频率变化导致动作速度改变。
- 每个阶段同时具备完成条件、连续稳定时间、最大持续时间和故障退出。
- 对关节角、关节速度、电机电流/力矩、机身角速度设置独立限制；任何一个越界都可中止。
- 倒扣与斜向姿态若没有经过验证的机械动作，先输出 UnsupportedDirection 并锁定，而不是猜测动作。

10. StandupSequence 设计

将现有 Phase 0–3 原样抽取为独立序列，第一轮只改变结构、不改变目标值和 PID。当前从普通起立、台阶退出、倒地恢复进入起立的入口语义不同，应改成显式 StartMode。

```
enum class StandupStartMode {
    Full,                // 从 Phase 0 完整执行
    RetractAtZero,       // 已由自起摆到交接姿态，从收腿/回零阶段开始
    StairExit             // 台阶退出专用容差与入口
};

struct SequenceResult {
    SequenceStatus status; // Idle/Running/Succeeded/Failed
    uint8_t phase;
    SequenceFault fault;
    VirtualChassisCommand command;
};
```

10.1 建议阶段条件

阶段	命令	完成条件（需连续满足）	必须新增
0 摆腿准备	腿摆角到 theta_init；轴向力按现有行为保持	左右腿角误差均在容差内	stable_ms、timeout_ms、关节有效
1 收腿	腿长到 low_length	左右腿长误差均在容差内	接触/电流异常处理
2 回零	腿长保持；theta_target 按速度斜坡到 0	目标到 0 且实际误差/速度满足	按 dt 限速；避免只看目标值
3 完成	输出安全交接命令	主 FSM 同时确认 upright	一次性 succeeded 事件与可查询状态

现有 standup_phase_stable_ticks_ 字段可转化为正式的 stable_ms 机制；原有来源 latch（如 standup_from_recovery_latch_）应由 StartMode 与主 FSM 上下文替代。

11. 模块接口与职责边界

模块	拥有的状态	不得承担的职责
PostureObserver	IMU 质量、坐标变换、最近样本质量	倒地阈值、FSM 转移、电机命令
FallDetector	候选计时、滞回、方向锁定	自起力矩、gimbal 控制、起立 phase
ChassisFsm	模式、失败锁定、重试计数	四元数数学、关节 PID、隐式局部静态计时
SelfRightSequence	自起阶段与阶段计时	决定是否允许恢复、直接写电机 CAN
StandupSequence	起立阶段与目标斜坡	判断倒地、决定主模式
TorqueComposer	限幅、VMC/Jacobian、最终命令	事件计时与恢复策略

11.1 推荐周期顺序

```

1. ReadSensors(now)
2. posture = posture_observer.Update(raw_imu, now)
3. leg_state = leg_observer.Update(encoders, motors, now)
4. fall = fall_detector.Update(posture, leg_state, mode_context, now)
5. fsm_output = chassis_fsm.Update(operator_request, fall, faults, now)
6. sequence_output = selected_sequence.Update(..., dt)
7. command = chassis_controller.Compose(fsm_output, sequence_output, ...)
8. ApplySafetyLimits(command)
9. WriteMotors(command)
10. PublishTelemetry(all_intermediate_values)

```

关键变化

FSM 与控制器消费的是同一周期已经计算好的观测结果；BuildChassisFsmInput 不再从上一周期 Chassis::UpdateOutput 反向读取 posture_valid，也不再持有 fall_start_ms。

12. 参数结构

将分散的 constexpr 参数按模块组成结构体，每个机器人 variant 提供一份完整配置。角度配置可保留度数便于评审，但启动时预计算余弦；运行时直接比较 tilt_cos。

```

struct FallDetectorParams {
    float fall_enter_angle_deg;
    uint32_t fall_confirm_ms;
    float severe_angle_deg;
    uint32_t severe_confirm_ms;
    float upright_exit_angle_deg;
    uint32_t upright_confirm_ms;
    float upright_gyro_max_rad_s;
    float accel_norm_tolerance_m_s2;
    uint32_t imu_stale_ms;
    float direction_dominance_ratio;
};

struct SelfRightParams { PhaseParams phase[5]; SafetyLimits limits; };
struct StandupParams    { PhaseParams phase[4]; float theta_ramp_rad_s; };

```

12.1 参数治理

- 参数名包含单位（_ms、_rad_s、_m、_deg），禁止再使用“每周期步长”而不注明频率。
- 公共默认值与 variant 覆盖分离；编译时检查进入阈值大于退出阈值、超时大于稳定时间等约束。
- 每次调参记录机器人编号、固件 commit、场地、负载、电池电压和日志编号。
- 清理 kLegRecoverThetaDotRampStep、kRecoveryGravityRampScale、kPitchBrake* 等参数前，先由静态引用与实机日志确认是否真正未消费。

13. 遥测与可观测性

重构是否可调，取决于能否看到每个决策中间量。建议在 debug 数据中增加以下字段，并保留当前 Euler 与 posture_valid 一段过渡期用于对比。

类别	字段
IMU 质量	quat_norm、quat_valid、imu_age_ms、frame_config_id、fault
姿态	up_body.x/y/z、tilt_cos、tilt_deg（调试）、gyro_body、accel_norm
检测	fall_candidate、severe_candidate、fall_confirmed、upright_confirmed、hold_ms、direction、cause
FSM	state、previous_state、transition_reason、state_elapsed_ms、retry_count、lockout_reason
序列	sequence、phase、phase_elapsed_ms、completion_error、command before/after limit
兼容对比	legacy_posture_valid、legacy_recovery_state、new_shadow_state、mismatch_reason

14. 分阶段迁移计划

推荐按“先观测、再影子判断、再拆结构、最后切控制”的顺序实施。每一步均可独立合入、记录数据和回滚。

阶段	工作内容	交付物	退出门槛
0 基线与标定	记录现有状态转移、动作阶段和典型日志；完成坐标约定与六面试验。	基线日志、FrameConvention、测试记录	六面方向全部正确；现有行为可复现
1 姿态观测影子接入	反馈接口加入四元数；实现 PostureObserver，只发布遥测，不影响控制。	posture_observer.*、debug 字段	长时间无 NaN/超时误报；yaw 不影响 tilt
2 检测器影子运行	实现 FallDetector，与 legacy posture_valid 并行，回放与实机采集误差。	fall_detector.*、对比工具/日志	目标场景无危险误触发；差异均可解释
3 数据流解环	调整 control.cc 顺序；FSM 使用同周期 Observation/Detection；移除 input.cc 局部倒地计时。	显式 Observation → FSM 数据流	单测证明无上一周期反馈；正常模式行为不变
4 抽取 StandupSequence	行为等价搬迁 Phase 0–3；加入 StartMode、稳定时间和阶段超时。	standup_sequence.*	旧/新命令离线对比一致；台阶入口通过
5 抽取 SelfRightSequence	行为等价搬迁前后/侧向/gimbal/腿动作；再加入方向锁定、dt ramp 与保护。	self_right_sequence.*	悬挂与低力矩试验通过；故障可中止
6 切换主判据	新 FSM 使用 fall_confirmed/upright_confirmed；加入 RecoveryFailed 锁定和有限重试。	生产路径切换开关、回滚开关	完整矩阵与实机阶段验收通过
7 清理与调参冻结	删除 legacy posture_valid 的控制语义、无用 latch/参数；整理多 variant 配置。	清理提交、参数报告、运维说明	连续场测通过；代码审查无双重路径

14.1 推荐提交拆分

1. 仅增加 QuaternionFeedback 与调试字段，不改控制行为。
2. 增加 PostureObserver 及单测，影子发布 up_body。
3. 增加 FallDetector 及回放测试，影子发布事件。
4. 重排观察—决策—控制顺序，保留 legacy detector 作为输入。

- 5. 抽取 StandupSequence，逐周期比较输出。
- 6. 抽取 SelfRightSequence，逐周期比较输出。
- 7. 新 FSM 与检测器接管；保留编译期开关用于一次版本回滚。
- 8. 数据充分后删除旧路径和过渡字段。

15. 文件级改动建议

文件/模块	建议改动
include/chassis/state.hpp	加入 QuaternionFeedback、BodyFrameImu 或等价结构；保留明确单位和时间戳。
include/actuators.hpp	读取 quat_w/x/y/z；将 IMU→车体固定变换集中化，逐步移除散落轴交换。
posture_observer.hpp/.cc（新）	归一化、数据质量、安装变换、up_body、低动态标志。
fall_detector.hpp/.cc（新）	滞回、持续时间、严重路径、方向锁定、直立确认、原因码。
self_right_sequence.hpp/.cc（新）	自起阶段、超时、安全限制、虚拟命令输出。
standup_sequence.hpp/.cc（新）	Phase 0-3、StartMode、dt ramp、稳定时间与超时。
control.cc	重排为 Observe→Detect→FSM→Sequence→Control；删除上一周期姿态反馈依赖。
input.cc / input.hpp	删除 fall_start_ms、was_posture_invalid 与倒地事件计时职责。
chassis_fsm.cc/.hpp	引入 RecoveryPending/Active/Failed；明确转移原因与锁定复位。
chassis.cc/.hpp	移除直接 posture invalid 大分支和隐式起立来源 latch；只做命令合成/底层控制。
include/params.hpp	改为 FallDetectorParams/SelfRightParams/StandupParams，按 variant 实例化。
debug.cc/.hpp	加入第 13 章遥测字段与 legacy/new 对比。

16. 测试与验证计划

16.1 单元测试

对象	必测用例	期望
四元数数学	identity、纯 yaw、 $\pm 90^\circ$ pitch/roll、倒扣、q/-q、非单位 q	up_body 符合解析值；yaw 不改变倾斜
异常输入	NaN、Inf、全零、极小范数、陈旧时间戳、跳变	明确 invalid/fault，不输出随机方向
安装变换	六面输入 + R_BS	全部映射到机体系期望轴
FallDetector	阈值抖动、持续时间不足、严重路径、退出滞回	无边界抖动；计时符合定义
方向分类	前/后/左/右/倒扣/斜向、q 符号翻转	确认时锁定；动作中不翻转
计时	tick wrap、控制周期抖动、丢帧	使用无符号差值/dt，行为稳定
FSM	候选消失、确认、传感器故障、超时、复位、重试	Pending 不动作；失败持续锁定
序列	每阶段完成/超时/关节故障/中止	输出状态与安全命令确定且可复现

16.2 日志回放场景

- 正常静止、低速、高速、急加速、急刹车、原地旋转。
- 跨坎、上下坡、台阶攀爬、跳跃起落、单轮冲击。
- 人工缓慢前趴/后躺/左右侧躺/倒扣/斜向倒地。
- 跌倒后弹跳、连续滚动、被外力扶正、恢复中再次倒下。
- IMU 掉线、冻结、四元数异常、加速度冲击、遥控失联、低电压。

16.3 实机递进

1. 电机断电：仅验证姿态、检测与 FSM 影子状态。
2. 悬挂架：电机上电但限制力矩，验证方向、阶段顺序和中止。
3. 软垫 + 安全绳：低力矩完成单方向动作，每次只开放一种姿态。

4. 全方向软垫：逐步提高动作限制，覆盖重复跌倒与失败锁定。
5. 目标场地：与正常行驶、台阶和比赛策略联动，验证不误触发。

安全要求

自起测试必须有急停、限力矩、人员隔离和机械约束。任何传感器故障、方向未知或阶段超时都应优先产生可预测的安全停止，而不是继续尝试“猜测性”翻身。

17. 验收标准、风险与回滚

17.1 功能验收

- 四元数与安装变换通过六面测试；纯 yaw 不改变 tilt_cos；q/-q 不改变检测结果。
- RecoveryPending 在确认时间内不产生自起动作；严重倒地快速路径符合配置。
- 直立退出同时满足机身倾角、腿构型、角速度、传感器有效和持续时间。
- 前/后/左/右动作方向与物理姿态一致；未知/倒扣策略明确且经过评审。
- 所有动作阶段均有完成、稳定、超时与故障出口；超时后 RecoveryFailed 持续锁定。
- 正常行驶、急加减速、旋转、跳跃和台阶测试中无不可接受误触发。
- 关闭新路径开关可回到已知 legacy 行为，回滚不需要修改传感器驱动。

17.2 主要风险

风险	后果	控制措施
四元数方向/安装变换错误	方向分类相反，自起动作危险	六面测试；唯一 R_BS；代码中禁止散落符号
阈值照搬欧拉角	正常动态误触发或倒地漏检	影子日志；按场景 ROC/混淆矩阵标定
结构与动作同时大改	无法定位回归原因	先行为等价抽取，再改变算法与参数
超时未持久化	反复启停或无限恢复	RecoveryFailed latch + 明确 reset/retry
只看 u_z	能判倾斜但动作方向错误/未知	确认后用 u_x/u_y 分类并锁定；斜向单独处理
只用加速度方向	急加速、碰撞、自由落体误判	仅低动态交叉校验，四元数为主
多 variant 参数漂移	某机型动作不安全	结构化配置、每机型验收矩阵、编译时约束

17.3 回滚策略

在阶段 1-6 保留编译期或受控运行期开关：

LegacyDetector/NewDetectorShadow/NewDetectorActive。动作序列切换应独立于检测器切换，以

便定位是“判错”还是“动作错”。出现方向不符、误触发、传感器质量异常或阶段超时率上升时，立即退回上一个已验收组合，保留日志而不现场临时修改符号。

18. 需要评审确认的决策

决策项	推荐默认	必须由谁/如何确认
机体系轴与四元数旋转方向	按 x 前/y 左/z 上统一	传感器负责人 + 六面实测
IMU 固定安装旋转 R_BS	常量矩阵/四元数集中配置	机械安装图 + 实测
倒地/直立阈值与时间	50°/28°、220/400 ms 起步	日志统计 + 实机测试
严重倒地快速路径	78°、80 ms 起步	安全评审 + 软垫验证
失败后策略	默认锁定，人工复位	比赛策略/安全负责人
倒扣与斜向策略	未验证前不自动动作	机械能力与动作专项测试
gimbal 让位超时	超时锁定或低风险降级	底盘/云台联合评审
台阶/跳跃模式门控	检测始终运行，动作按模式许可	任务状态机负责人

19. 推荐实施顺序与工作量焦点

最优先的三件事

① 六面验证四元数约定和安装变换；② 影子接入 PostureObserver/FallDetector 并采集日志；③ 消除上一周期 chassis_control_output → FSM 的反馈。完成这三项后，再抽取和切换动作序列。

该顺序把最高风险的“方向是否正确”和“检测是否误触发”放在电机动作之前解决，也使后续每次代码审查都能围绕单一职责展开。不要在第一版同时重调全部自起力矩；先证明新架构能够逐周期复现旧动作，再针对不同方向逐项优化。

附录 A：公式与测试速查

A.1 直接公式

```
q = [w,x,y,z], normalize(q)

up_body.x = 2(xz - wy)
up_body.y = 2(yz + wx)
up_body.z = 1 - 2(x2 + y2)

tilt_cos = up_body.z
tilt = acos(clamp(tilt_cos, -1, 1))
fall when tilt > enter_angle ⇔ tilt_cos < cos(enter_angle)
upright when tilt < exit_angle ⇔ tilt_cos > cos(exit_angle)
```

A.2 解析测试姿态（无安装偏转时）

旋转示例	四元数 [w,x,y,z]	期望 up_body
单位姿态	[1,0,0,0]	[0,0,1]
任意纯 yaw ψ	$[\cos \psi/2, 0, 0, \sin \psi/2]$	[0,0,1]
绕 x +90°	$[\sqrt{2}/2, \sqrt{2}/2, 0, 0]$	[0,1,0]
绕 x -90°	$[\sqrt{2}/2, -\sqrt{2}/2, 0, 0]$	[0,-1,0]
绕 y +90°	$[\sqrt{2}/2, 0, \sqrt{2}/2, 0]$	[-1,0,0]
绕 y -90°	$[\sqrt{2}/2, 0, -\sqrt{2}/2, 0]$	[1,0,0]
绕 x 180°	[0,1,0,0]	[0,0,-1]

以上符号基于本文件第 4.3 节的 R_WB/Hamilton 约定。若设备输出相反旋转或机体系轴不同，应通过 R_BS/共轭统一到项目约定，不能修改方向分类代码去“补符号”。

附录 B：当前代码关键定位

主题	路径与位置（以当前工作区为准）
姿态有效判定	app/targets/wheel_legged/chassis.cc:307–318
姿态无效恢复入口	app/targets/wheel_legged/chassis.cc:340 起
起立 Phase 0–3	app/targets/wheel_legged/chassis.cc:382–453
恢复动作主要分支	app/targets/wheel_legged/chassis.cc:678–871
倒地计时/FSM 输入	app/targets/wheel_legged/input.cc:739–776
控制周期与上一周期输出	app/targets/wheel_legged/control.cc:369, 420–446
恢复 FSM 转移	app/targets/wheel_legged/chassis_fsm.cc:439–471
FSM 参数	app/targets/wheel_legged/include/params.hpp（各 variant 的 chassis_fsm/chassis）
IMU 四元数访问器	libs/librm/src/librm/device/sensor/hipnuc_imu.hpp:95–98
CAN IMU 四元数	libs/librm/src/librm/device/sensor/hipnuc_imu_can.hpp:158–161

代码位置用于评审导航；后续重构会移动行号。建议在实施提交中以函数名、类型名和测试名建立长期可追踪关系。

附录 C：实施检查清单

- ☐ FrameConvention 已评审并通过六面测试
- ☐ QuaternionFeedback 带时间戳、单位和有效标志
- ☐ PostureObserver 无控制副作用且有异常输入测试
- ☐ FallDetector 有滞回、普通/严重持续时间和方向锁定
- ☐ 加速度计仅在低动态窗口参与一致性校验
- ☐ leg_configuration_safe 与 body_upright 分离
- ☐ RecoveryPending 不输出自起动作
- ☐ RecoveryFailed 可持久锁定并记录原因

- ☐ SelfRightSequence 与 StandupSequence 均有阶段超时
- ☐ 所有 ramp 使用 dt 和物理单位/秒
- ☐ FSM 消费同周期观测，不再读上一周期姿态输出
- ☐ 新旧路径影子对比完成并保存日志
- ☐ 悬挂、软垫、安全绳与目标场地分级验收完成
- ☐ 每个机器人 variant 均有独立参数与测试记录
- ☐ 回滚开关经过实际验证

评审建议

首次评审只需冻结三项：坐标/四元数约定、模块职责边界、迁移阶段与安全门槛。动作力矩和最终阈值应在影子数据与低风险实机测试后单独评审。